VERIQTA

PLATFORM ENGINEERING

FOR BEGINNERS

FROM DEVOPS FOUNDATIONS TO YOUR FIRST INTERNAL DEVELOPER PLATFORM

WHAT YOU WILL LEARN

Linux | Networking | Git | Cloud | IaC | Containers | Kubernetes
CI/CD | GitOps | Developer Portals | Backstage | Security
Observability | Self-Service | Troubleshooting | Platform Design
Build Your First Internal Developer Platform

BUILD PLATFORMS. EMPOWER DEVELOPERS. DELIVER SOFTWARE FASTER TOGETHER.

@veriqta | veriqta | veriqta | veriqta | @veriqta | @veriqta

WHAT IS PLATFORM ENGINEERING?

PLATFORM ENGINEERING IN SIMPLE LANGUAGE

Platform engineering is the practice of building and operating self-service tools, workflows, and infrastructure that enable developers to build, test, and ship software with ease.

WHY THE DISCIPLINE EXISTS

- Infrastructure is complex.
- Developers waste time on undifferentiated work.
- Manual processes don't scale.
- Teams build things differently.
- Security, compliance, and reliability must be built-in.

PROBLEMS IT SOLVES

- Reduces developer friction
- Speeds up software delivery
- Improves consistency & reliability
- Enforces security & compliance
- Reduces operational toil
- Creates reusable, scalable systems

DEVELOPERS AS THE PLATFORM'S CUSTOMERS

- Developers are the primary users.
- The platform team builds for them.
- The platform should be easy, reliable, documented, and self-service.
- Good platforms empower developers, they don't block them.

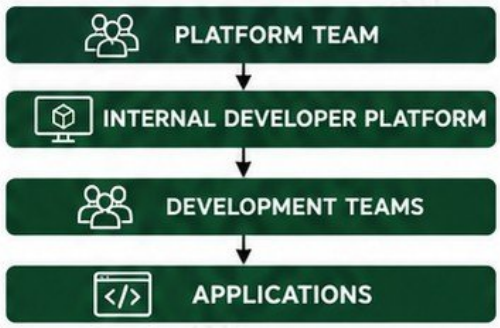
PLATFORM ENGINEERING vs DEVOPS vs SRE

FOCUS AREA	PLATFORM ENGINEERING	DEVOPS	SRE
Primary Goal	Build internal developer platforms and self-service capabilities.	Improve collaboration and automate the software delivery lifecycle.	Ensure reliability, availability, and performance in production.
Who They Serve	Development teams (platform as a product)	Everyone in the delivery pipeline (dev + ops + tools)	End users and the business (reliability of services)
What They Build	Self-service tools, templates, golden paths, portals, platforms.	Pipelines, automation, CI/CD, infrastructure as code.	Monitoring, alerting, reliability systems, incident response.
Success Looks Like	Developers can do more themselves, faster and safely.	Frequent, reliable, and automated software delivery.	Systems are reliable, resilient, and observable.
Key Mindset	Product thinking for internal systems.	Automation and collaboration at every step.	Reliability engineering and continual improvement.

WHAT DOES A PLATFORM ENGINEER ACTUALLY DO?

- Builds and maintains self-service platforms
- Creates reusable templates and workflows
- Automates infrastructure provisioning
- Builds and maintains CI/CD and GitOps systems
- Ensures security, compliance, and governance
- Builds developer portals and documentation
- Integrates tools for a smooth developer experience
- Observes and improves platform reliability
- Works closely with developers to solve real problems

MENTAL MODEL



JUNIOR ENGINEER TIP

Platform engineering is more than managing Kubernetes.
It is about building great developer experiences through tools, automation, and self-service systems.

THE PROBLEM PLATFORM ENGINEERING SOLVES

THE CHALLENGES TODAY

DEVELOPERS MANAGING TOO MUCH INFRASTRUCTURE
Developers spend time on infrastructure instead of building features.

REPETITIVE SETUP WORK
Teams constantly repeat environment setup, configurations, and integrations.

DIFFERENT DEPLOYMENT PROCESSES BETWEEN TEAMS
Every team deploys differently, causing inconsistency and confusion.

INFRASTRUCTURE COMPLEXITY
Too many moving parts, dependencies, and manual steps.

SECURITY AND CONFIGURATION INCONSISTENCIES
Misconfigurations, over-permissions, and policy violations create risk.

TOO MANY TOOLS DEVELOPERS MUST UNDERSTAND
Multiple tools, dashboards, and systems slow developers down.

THE "YOU BUILD IT, YOU CONFIGURE EVERYTHING" PROBLEM
Developers are forced to become system administrators.

WHAT THIS CAUSES

- Slower feature delivery
- More errors and failed deployments
- Inconsistent environments
- High operational overhead
- Poor developer experience
- Security and compliance risks
- Teams reinventing the same solutions

HOW PLATFORMS CREATE A PAVED ROAD

- Self-service infrastructure
- Standardized, reusable workflows
- Consistent environments
- Built-in security and compliance
- Fewer tools to learn
- Automation everywhere
- Developers focus on code, not plumbing

WITHOUT A PLATFORM

CONFUSING. SLOW. RISKY.

VS

WITH A PLATFORM

SIMPLE. FAST. SAFE.

JUNIOR ENGINEER TIP

A platform removes complexity from the developer's path.
It gives them a smooth, secure, and reliable way to build and ship software.

UNDERSTANDING INTERNAL DEVELOPER PLATFORMS

BUILDING THE PAVED ROAD FOR DEVELOPERS

WHAT IS AN INTERNAL DEVELOPER PLATFORM (IDP)?

An Internal Developer Platform (IDP) is a set of self-service tools, services, and workflows that help developers build, test, deploy, and operate software without dealing with undifferentiated infrastructure.

PLATFORM vs PORTAL

PLATFORM	PORTAL
Provides capabilities and services	User interface to discover and interact
Includes workflows, automation, and infrastructure	Surface area of the platform
Automates and enforces standards	Helps developers find what they need
Enables self-service actions	Does not do the work itself
Solves real developer problems	A part of the IDP, not the whole thing

PLATFORM CAPABILITIES

SELF-SERVICE INFRASTRUCTURE
Developers can provision resources like databases, buckets, queues, and more without tickets.

DEPLOYMENT CAPABILITIES
Standardized pipelines, deployment strategies, and release management built-in.

ENVIRONMENT CREATION
On-demand environments for feature branches, tests, QA, and previews.

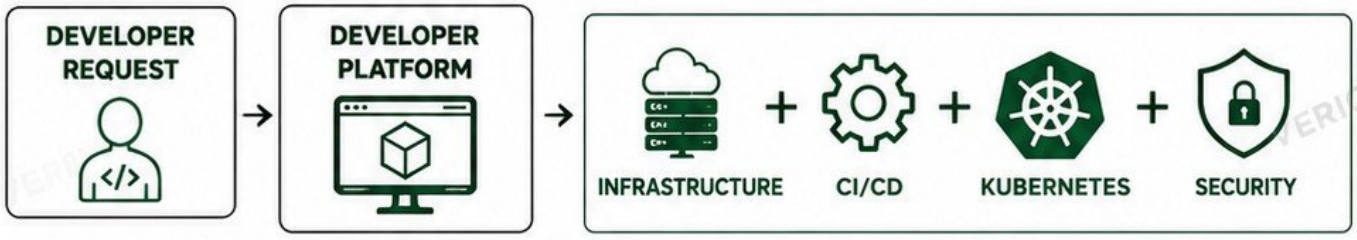
OBSERVABILITY
Logs, metrics, traces, alerts, and dashboards out of the box.

SECURITY CONTROLS
Built-in security, compliance, policies, secrets management, and guardrails.

DOCUMENTATION
Centralized docs, runbooks, APIs, and best practices for developers.

GOLDEN PATHS
Pre-approved, opinionated workflows and templates so teams move fast without breaking standards.

HOW AN IDP WORKS (EXAMPLE)



WHY IT MATTERS

- Developers focus on building product, not plumbing.
- Faster delivery with fewer errors.
- Consistent, secure, and compliant by default.
- Reusable services and automation reduce toil.
- Better developer experience and productivity.
- Scale teams without adding operational chaos.

THE PLATFORM ENGINEER'S TECHNOLOGY STACK

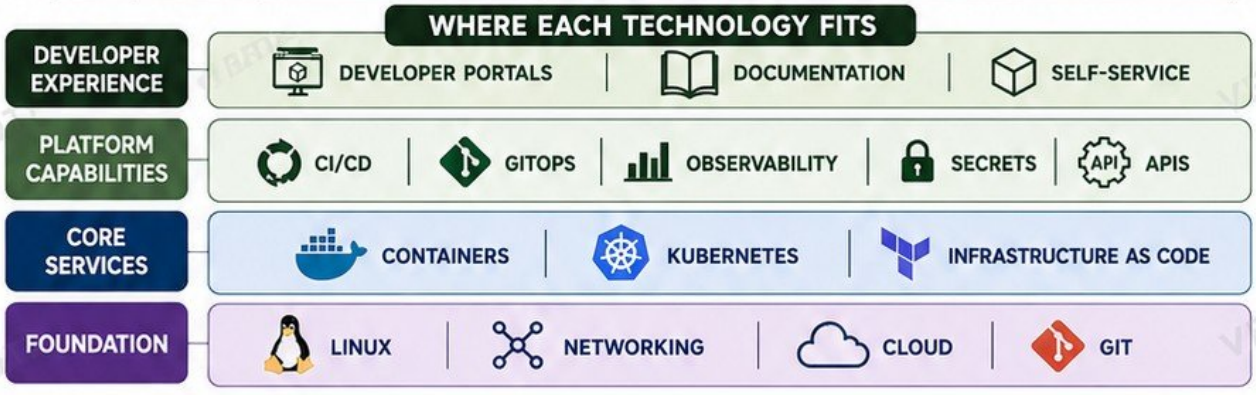
INTRODUCE THE ECOSYSTEM BEFORE GOING DEEPER

THE ECOSYSTEM AT A GLANCE

Platform engineering brings multiple technologies together to deliver a self-service, secure, and reliable developer experience.

1	LINUX	The foundation. Operating systems power servers, containers, and everything in between.
2	NETWORKING	Connects systems and services. Includes DNS, IP, ports, routing, load balancers, firewalls, and more.
3	GIT	Version control for code, infrastructure, and configurations. The source of truth for everything.
4	CLOUD	Provides compute, storage, databases, networking, and managed services on demand.
5	INFRASTRUCTURE AS CODE	Automates infrastructure provisioning. Tools like Terraform ensure consistency and repeatability.
6	CONTAINERS	Standardize applications and dependencies into portable, isolated units.
7	KUBERNETES	Orchestrates containers at scale. Manages deployments, scaling, networking, and self-healing.
8	CI/CD	Automates build, test, and delivery of applications. Ensures quality and speed.
9	GITOPS	Declarative delivery using Git as the single source of truth. Automates deployments and reduces drift.
10	OBSERVABILITY	Provides visibility into systems and applications using logs, metrics, traces, alerts, and dashboards.
11	SECRETS	Safely store and manage sensitive data like passwords, tokens, and API keys.
12	DEVELOPER PORTALS	The front door for developers. Discover services, request resources, read docs, and track ownership.
13	APIS	The glue that connects everything. Platforms expose APIs for automation, integration, and self-service.

WHERE EACH TECHNOLOGY FITS



KEY TAKEAWAY

Platform engineering is not about one tool. It's about how these technologies work together to create a reliable, secure, and easy path for developers to deliver software.

LINUX FOUNDATIONS FOR PLATFORM ENGINEERS

THE OPERATING SYSTEM IS THE FOUNDATION OF EVERY PLATFORM

CORE LINUX CONCEPTS

FILES AND DIRECTORIES
Everything in Linux is a file. Organized in a hierarchical directory structure.

USERS AND GROUPS
Control who can access the system and what they can do.

PERMISSIONS
Define who can read, write, or execute files and directories.

PROCESSES
Running programs and tasks. Linux is a multi-tasking system.

SERVICES
Background processes that keep your system and applications running.

ENVIRONMENT VARIABLES
Store configuration values that affect system and application behavior.

LOGS
Record events, errors, and activities. Essential for troubleshooting.

SSH
Secure access to remote systems. A must-have skill for platform engineers.

IMPORTANT COMMANDS

`pwd`	**PRINT WORKING DIRECTORY** Shows your current location. Example: `pwd`
`ls`	**LIST DIRECTORY CONTENTS** Lists files and folders. Example: `ls -lah`
`cd`	**CHANGE DIRECTORY** Move between directories. Example: `cd /var/log`
`ps`	**PROCESS STATUS** Shows running processes. Example: `ps aux`
`top`	**INTERACTIVE PROCESS VIEWER** Real-time view of system processes. Example: `top`
`systemctl`	**MANAGE SYSTEM SERVICES** Start, stop, enable, check services. Example: `systemctl status nginx`
`journalctl`	**VIEW SYSTEM LOGS** Query and follow system logs. Example: `journalctl -xe`

HOW LINUX POWERS PLATFORMS

LINUX
Provides the runtime environment

→

SERVICES
Run essential platform components

→

APPLICATIONS
Deployed on top of Linux services

→

SECURITY
Protected by Linux security model

→

RELIABILITY
Monitored and kept healthy by Linux

WHY PLATFORMS ULTIMATELY DEPEND ON OPERATING SYSTEMS

Kubernetes, containers, databases, and every platform service run on Linux. Mastering Linux gives you the power to build, secure, monitor, and troubleshoot reliable platforms from the ground up.

NETWORKING FOUNDATIONS

UNDERSTAND THE REQUEST PATH. FIX THE RIGHT LAYER.

THE REQUEST PATH

DEVELOPER
Starts the request
(Browser, App, CLI)

↓

DNS
Translates domain name
to IP address

↓

IP ADDRESS
Unique address that
identifies a server

↓

LOAD BALANCER
Distributes traffic across
healthy instances

↓

APPLICATION
Processes the request
and returns a response

↓

DATABASE
Stores and retrieves
the application data

KEY NETWORKING CONCEPTS

IP ADDRESSES	Every device on a network has a unique IP. IPv4 (e.g., 192.168.1.10) or IPv6.
DNS	Maps human-readable names (example.com) to IP addresses.
PORTS	Logical endpoints on a server. Examples: 80 (HTTP), 443 (HTTPS), 22 (SSH).
TCP	Connection-oriented protocol. Reliable, ordered, and error-checked.
HTTP / HTTPS	Application layer protocols for web traffic. HTTPS is HTTP over TLS (encrypted).
ROUTING	Moves packets from one network to another using routers and gateways.
FIREWALLS	Control traffic in and out based on rules. Protect your systems.
LOAD BALANCERS	Distribute traffic, improve availability, and perform health checks.
PRIVATE vs PUBLIC NETWORKS	Private: internal, not directly accessible. Public: accessible from the internet.

ESSENTIAL NETWORKING COMMANDS

`curl`	Send HTTP requests and see responses.	`curl https://example.com`
`ping`	Check connectivity to a host.	`ping 8.8.8.8`
`dig`	Query DNS records and troubleshoot DNS.	`dig example.com`
`ss`	Show open sockets and listening ports.	`ss -tuln`

JUNIOR ENGINEER TIP

Always test each layer of the request path.
Fix the layer that is broken,
not the one above or below it.

TROUBLESHOOTING "THE APPLICATION IS UNREACHABLE"

1. **CHECK THE SYMPTOM**
   What exactly fails? Browser error? Timeout? Connection refused?
2. **VERIFY DNS RESOLUTION**
   Use dig or nslookup. Does the domain resolve to the right IP?
3. **TEST CONNECTIVITY**
   Use ping to test reachability to the IP address.
4. **CHECK PORT ACCESS**
   Use ss or telnet/curl to verify the target port is reachable.
5. **VERIFY ROUTING**
   Ensure the correct route exists to the destination network.
6. **CHECK FIREWALLS / SECURITY GROUPS**
   Confirm inbound/outbound rules allow the traffic.
7. **CHECK LOAD BALANCER / SERVICE**
   Is the target healthy? Is the listener configured correctly?
8. **CHECK THE APPLICATION**
   Is the app running? Are there errors in logs?
9. **REVIEW LOGS AND METRICS**
   Use logs, metrics, and traces to find the root cause.

GIT AND VERSION CONTROL

THE FOUNDATION OF MODERN INFRASTRUCTURE AND PLATFORM ENGINEERING

CORE CONCEPTS

REPOSITORY
A project folder that contains all your files and their history.

COMMIT
A snapshot of changes with a message that explains why.

BRANCH
A separate line of development. Work without affecting others.

PULL REQUEST
Ask to merge your changes into another branch. Start a review.

MERGE
Combine changes after review and approval.

MERGE CONFLICTS
Git can't auto-merge changes. You must resolve and commit the fix.

WHY INFRASTRUCTURE BELONGS IN GIT

- Version history for every change.
- Traceability and accountability.
- Rollback to any previous state.
- Collaboration through pull requests.
- Code review improves quality.
- Automation and CI/CD become possible.
- Foundation for GitOps and declarative delivery.
- Auditable and compliant by default.

INFRASTRUCTURE CHANGES THROUGH PULL REQUESTS

ENGINEER
Create branch and make changes.

PUSH
Push branch to remote repository.

PULL REQUEST
Open PR for review.

CODE REVIEW
Team reviews the changes.

MERGE
Approved changes are merged.

CODE REVIEW: PREVENT PROBLEMS BEFORE THEY REACH PRODUCTION

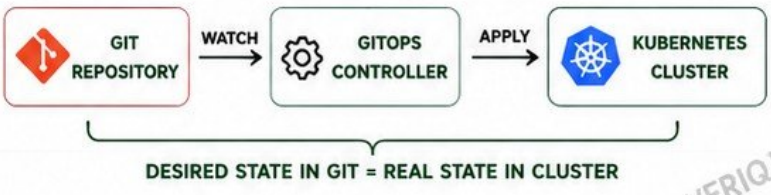
- Ensures changes follow standards and best practices.
- Helps catch mistakes early.
- Shares knowledge across the team.
- Builds a culture of quality and collaboration.

JUNIOR ENGINEER TIP

Good reviews are about the code, the design, the security, and the impact on production.

GIT AS THE FOUNDATION OF GITOPS

- Desired state is stored in Git.
- Git is the single source of truth.
- GitOps tools watch the repo.
- Changes are automatically applied to Kubernetes.
- Auditable, predictable, and reliable.



REMEMBER

If it's not in Git, it's not versioned. If it's not versioned, it's not reliable.

COMMON BEGINNER MISTAKES

- Committing secrets to Git.
- Pushing directly to main branch.
- Skipping code review.
- Large, undocumented changes.
- Ignoring merge conflicts.

GIT ISN'T JUST FOR CODE. IT'S FOR INFRASTRUCTURE, SECURITY, AUTOMATION, AND THE FUTURE OF YOUR PLATFORM.

CLOUD INFRASTRUCTURE FOUNDATIONS

THE BUILDING BLOCKS OF MODERN CLOUD PLATFORMS

CORE CLOUD SERVICES

COMPUTE
Run your applications on virtual machines, containers, or serverless platforms.

STORAGE
Store data durably and scalably. Object storage, block storage, and file storage.

NETWORKING
Virtual networks, subnets, IP addressing, routing, firewalls, and private connectivity.

DATABASES
Managed databases for relational, NoSQL, caching, and analytics workloads.

IDENTITY
Manage users, roles, permissions, and access across your cloud resources.

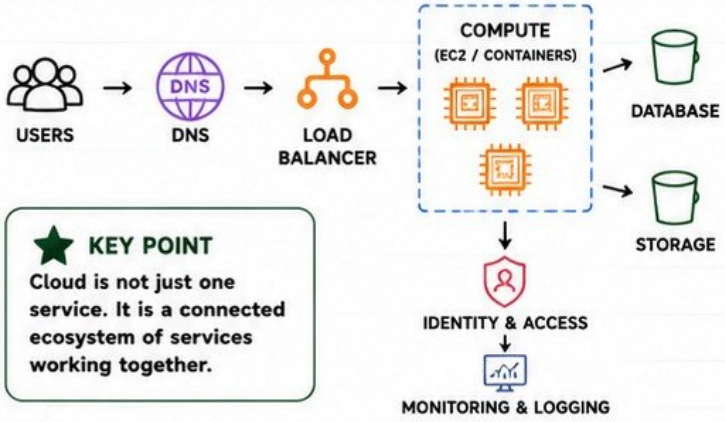
LOAD BALANCING
Distribute traffic across multiple targets for high availability and scalability.

DNS
Translate domain names to IP addresses. Route users to the right resources.

MONITORING
Collect metrics, logs, and alerts to understand health, performance, and usage.

REGIONS & AVAILABILITY ZONES
Regions are physical locations. Availability Zones isolate failure domains within a region.

HOW CLOUD SERVICES WORK TOGETHER



KEY POINT
Cloud is not just one service. It is a connected ecosystem of services working together.

SHARED RESPONSIBILITY MODEL

CLOUD PROVIDER IS RESPONSIBLE FOR	YOU (CUSTOMER) ARE RESPONSIBLE FOR
✔ Global Infrastructure	✔ Your Data
✔ Regions & AZs	✔ Access Management
✔ Physical Security	✔ Applications
✔ Hardware	✔ Operating Systems
✔ Networking	✔ Network Configuration
✔ Core Services	✔ Security Groups
✔ Availability	✔ Patching
✔ Fault Tolerance	✔ Monitoring & Alerts

WHY PLATFORM ENGINEERS MUST UNDERSTAND CLOUD ARCHITECTURE

- Design reliable, scalable, and secure platforms.
- Make informed decisions and choose the right services.
- Troubleshoot issues effectively across the stack.
- Automate and standardize infrastructure.
- Optimize cost, performance, and availability.

JUNIOR ENGINEER TIP

Platform engineers do not just "click in the console". You design, automate, secure, observe, and operate cloud systems that developers rely on.

**UNDERSTAND THE BUILDING BLOCKS.
THEN YOU CAN BUILD GREAT PLATFORMS.**

INFRASTRUCTURE AS CODE

AUTOMATE. STANDARDIZE. VERSION. REPEAT.

1 WHAT IS INFRASTRUCTURE AS CODE (IaC)?

Infrastructure as Code (IaC) means managing and provisioning infrastructure through code instead of manual processes. It makes infrastructure repeatable, consistent, versioned, and automated.

2 MANUAL vs AUTOMATED INFRASTRUCTURE

MANUAL INFRASTRUCTURE	AUTOMATED INFRASTRUCTURE (IaC)
✗ Click through consoles	✓ Code-driven and repeatable
✗ Time-consuming	✓ Consistent and reliable
✗ Prone to human error	✓ Version controlled
✗ Inconsistent configurations	✓ Peer reviewed
✗ Hard to reproduce	✓ Automated and scalable
✗ No version control	✓ Easy to audit and track
✗ Difficult to scale	✓ Safe to experiment

3 TERRAFORM INTRODUCTION

Terraform is a popular IaC tool that lets you define and manage infrastructure across multiple cloud providers using a simple, declarative language (HCL).

KEY TERRAFORM CONCEPTS

PROVIDERS
Plugins that allow Terraform to talk to cloud platforms like AWS, Azure, GCP, and more.

RESOURCES
The infrastructure objects you create (VMs, VPCs, Buckets, Databases, etc).

VARIABLES
Input values that make your configuration dynamic and reusable.

OUTPUTS
Return values from your infrastructure (IPs, IDs, URLs, endpoints, etc).

STATE
Terraform keeps track of what it created in a state file to manage changes.

4 TERRAFORM CORE COMMANDS

terraform plan

Preview the changes Terraform will make to your infrastructure. No changes are applied.

terraform apply

Apply the changes defined in your code to create, update, or delete infrastructure. Approve the plan and make it real.

5 WHY PLATFORM TEAMS CREATE REUSABLE MODULES

- ✓ Encapsulate best practices.
- ✓ Promote consistency across environments.
- ✓ Reduce duplication and human error.
- ✓ Speed up infrastructure provisioning.
- ✓ Make complex infrastructure simple to reuse.
- ✓ Easier maintenance and updates.
- ✓ Enable self-service for developers.
- ✓ Enforce security, compliance, and guardrails.

6 THE IaC WORKFLOW

GIT — Your code and infrastructure live in version control.

TERRAFORM — Terraform reads your code and plans or applies changes.

CLOUD API — Terraform communicates with the cloud provider API to provision or modify resources.

INFRASTRUCTURE — Your infrastructure is created, updated, and managed.

7 KEY TAKEAWAY

Infrastructure as Code brings engineering discipline to infrastructure.
Code it. Version it. Review it. Automate it.
That is how reliable platforms are built.

CODE YOUR INFRASTRUCTURE. CONTROL YOUR PLATFORM. SCALE WITH CONFIDENCE.

CONTAINERS AND DOCKER

PACKAGE ONCE. RUN ANYWHERE.

1 WHY CONTAINERS EXIST

- Ensure consistency across environments.
- Eliminate "it works on my machine".
- Lightweight and fast startup.
- Package the application and all dependencies.
- Run anywhere: laptop, server, cloud, Kubernetes.

2 IMAGE vs CONTAINER

IMAGE

A read-only template with everything needed to run an application.

CONTAINER

A running instance of an image. It has state and can be started, stopped, removed.

3 DOCKERFILE

A Dockerfile is a blueprint for building a Docker image.

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8080
CMD ["python", "app.py"]  # Run app
```

4 CONTAINER REGISTRY

- Stores and distributes container images.
- Public: Docker Hub, GHCR, ECR Public.
- Private: AWS ECR, GCR, ACR, Harbor.
- Images are versioned by tags.
- Example: myapp:1.0.0

5 PORTS

Expose container ports to allow traffic in/out.

`-p 8080:8080`

6 ENVIRONMENT VARIABLES

Store configuration outside the image.

`-e APP_ENV=prod`

7 VOLUMES

Persist data beyond the life of a container.

`-v /data:/app/data`

8 CONTAINER NETWORKING

Containers communicate over isolated networks.

`--network mynet`

9 BUILD ⟶ PUSH ⟶ RUN WORKFLOW

BUILD IMAGE

```
docker build -t
myapp:1.0.0 .
```

PUSH IMAGE

```
docker push
myapp:1.0.0
```

PULL IMAGE

```
docker pull
myapp:1.0.0
```

RUN CONTAINER

```
docker run -d -p 8080:8080 \
--name myapp myapp:1.0.0
```

10 WHY PLATFORMS STANDARDIZE APPLICATION PACKAGING

- Consistency across all environments.
- Faster onboarding for developers.
- Simplifies CI/CD and deployments.
- Improves security and compliance.
- Makes scaling and rolling updates easy.
- Works seamlessly with Kubernetes and orchestration.

JUNIOR ENGINEER TIP

Containers package your application. Platforms run, scale, secure, monitor, and manage them.

REMEMBER: Docker builds the image. You run the container. Platforms orchestrate everything at scale.

KUBERNETES
FOR PLATFORM ENGINEERS
THE FOUNDATION OF MODERN APPLICATION PLATFORMS

1 KUBERNETES CORE CONCEPTS

CLUSTER
A set of machines that run containerized applications.

CONTROL PLANE
Manages the cluster and makes global decisions.

WORKER NODES
Machines that run your containerized workloads.

PODS
The smallest deployable units in Kubernetes.

DEPLOYMENTS
Manages ReplicaSets and performs rolling updates.

SERVICES
Provides a stable endpoint to access your application.

CONFIGMAPS
Store non-sensitive configuration data as key-value pairs.

SECRETS
Store sensitive data like passwords, tokens, and keys.

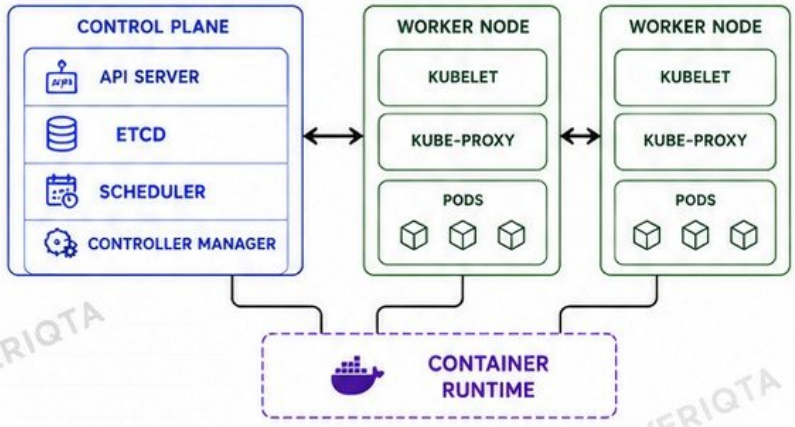
NAMESPACES
Logical isolation for resources within a cluster.

INGRESS / GATEWAY
Manages external access to services (HTTP/HTTPS).

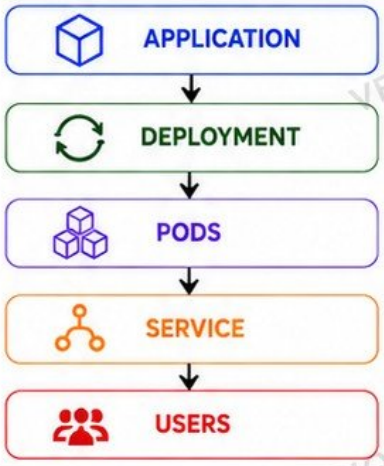
2 WHY KUBERNETES IN APPLICATION PLATFORMS?

- Standardizes how applications are deployed and operated.
- Provides scalability, resilience, and self-healing.
- Enables rolling updates and rollbacks.
- Supports multi-tenancy through namespaces.
- Integrates with CI/CD, observability, and security tools.
- Runs everywhere: on-prem, cloud, or hybrid.
- Forms the core of modern internal developer platforms.

3 HOW KUBERNETES ARCHITECTURE WORKS



4 APPLICATION FLOW IN KUBERNETES



5 KEY TAKEAWAYS

- Kubernetes orchestrates containers at scale.
- It provides the platform for resilient, modern applications.
- Understanding Kubernetes is essential for platform engineers.
- You don't just deploy to Kubernetes; you build platforms on it.

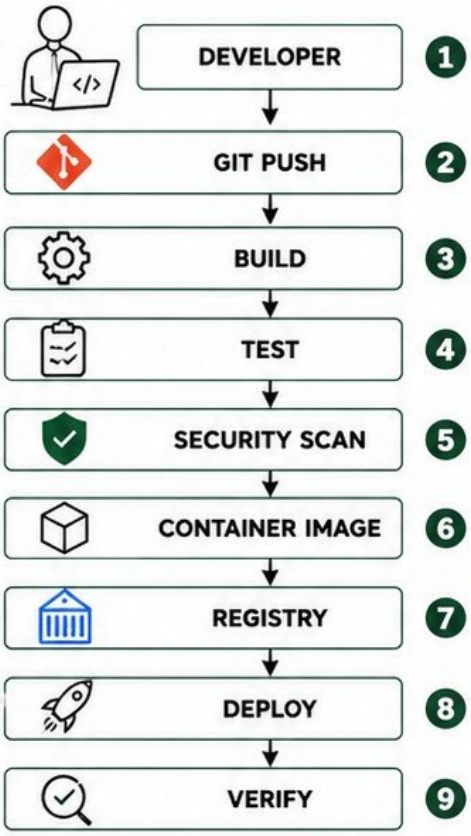
JUNIOR ENGINEER TIP

Learn the fundamentals deeply. Complex platforms are built on a strong Kubernetes foundation.

KUBERNETES IS NOT JUST A TOOL.
IT IS THE OPERATING SYSTEM FOR YOUR APPLICATION PLATFORM.

CI/CD AND THE SOFTWARE DELIVERY PATH

CI/CD, Continuous Integration and Continuous Delivery, is the automated process that takes code from your computer to a running application in production



KEY PARTS OF A PIPELINE

PIPELINES
A series of automated steps executed in order to deliver your application.

BUILD STAGES
Compile code, install dependencies, and create build outputs.

TESTS
Unit tests, integration tests, and quality checks to ensure code works.

ARTIFACTS
The outputs of the pipeline such as binaries, packages, or container images.

DEPLOYMENT
Release the application to an environment such as dev, staging, or production.

ROLLBACK
Return to a previous stable version quickly when something goes wrong.

WHAT HAPPENS AT EACH STAGE?

1. **DEVELOPER:** Write code and push to Git.
2. **GIT PUSH:** Triggers the pipeline automatically.
3. **BUILD:** Compile code and create artifacts.
4. **TEST:** Run automated tests to catch issues early.
5. **SECURITY SCAN:** Scan dependencies and image for vulnerabilities.
6. **CONTAINER IMAGE:** Package app in a container.
7. **REGISTRY:** Store the image for deployment.
8. **DEPLOY:** Deploy to the target environment.
9. **VERIFY:** Confirm the application is healthy.

WHY PLATFORM TEAMS PROVIDE REUSABLE PIPELINE TEMPLATES

- **CONSISTENCY:** Every team follows the same proven process.
- **SPEED:** Developers ship faster without building pipelines from scratch.
- **BEST PRACTICES:** Security, testing, and quality checks are built in.
- **REDUCED ERRORS:** Fewer mistakes, fewer failed deployments.
- **MAINTAINABILITY:** Platform team updates templates, all teams benefit.
- **FOCUS:** Developers focus on building features, not managing pipelines.

JUNIOR ENGINEER TIP

A good pipeline is reliable, observable, and repeatable.
Automate everything you can, and always verify before promotion.

GITOPS AND DECLARATIVE DELIVERY

Use Git as the single source of truth. The system automatically makes the cluster match what is defined in Git.

1 WHAT IS GITOPS?

- GitOps is an operational model that uses Git repositories as the source of truth for infrastructure and applications.
- Changes are made in Git, reviewed, and then automatically applied to the cluster.

2 CORE CONCEPTS

DESIRED STATE: The configuration stored in Git defines how the system should look.

GIT AS SOURCE OF TRUTH: Git is the single, versioned record of the desired state.

CONTROLLERS: Tools like Argo CD or Flux constantly check Git and the cluster.

RECONCILIATION: The controller automatically makes the cluster match the desired state in Git.

DRIFT: When the actual cluster state is different from what is defined in Git.

3 GITOPS TOOLS

ARGO CD

- Kubernetes native
- UI for managing apps
- Strong RBAC support
- Health and sync status

FLUX

- Lightweight
- Git native
- Automation focused
- Great for large scale

4 PULL-BASED DEPLOYMENT

- GitOps controllers pull changes from Git.
- No need to push deployments.
- Safer, auditable, and more secure.

5 GITOPS vs TRADITIONAL CI DEPLOYMENT

TRADITIONAL CI DEPLOYMENT	GITOPS (DECLARATIVE DELIVERY)
Push-based	Pull-based
CI for mangioys directly	Controllers pull and reconcile
State stored in CI system	State stored in Git
Harder to audit changes	Easy to audit via Git history
Manual drift detection	Automatic drift detection
Rollback through pipeline	Rollback through Git revert

6 GITOPS FLOW

GIT REPOSITORY
Holds the desired state (manifests, values, configs)

GITOPS CONTROLLER
Argo CD or Flux watches the repository.

KUBERNETES CLUSTER
Controller applies changes to the cluster.

DESIRED STATE
The cluster matches the state defined in Git.

7 BENEFITS OF GITOPS

- Single source of truth
- Automatic reconciliation
- Auditability and history
- Consistency across environments
- Self-healing
- Safer and more secure
- Supports Git workflows
- Scales with your organization

JUNIOR ENGINEER TIP

If it is not in Git, it does not exist. Use Git for everything. Let the controller keep your cluster in the desired state.

SELF-SERVICE AND GOLDEN PATHS

This is where the handbook becomes distinctly Platform Engineering.

1 WHAT IS SELF-SERVICE?

- Self-service means developers can get the infrastructure and tools they need without opening a ticket.
- It is fast, consistent, and available on demand.
- Developers focus on building features, not waiting for approvals.

2 WHAT IS A GOLDEN PATH?

- A golden path is the recommended, secure, and supported way to build and deploy applications.
- It removes guesswork and prevents bad or risky configurations.
- It helps teams move fast without breaking standards.

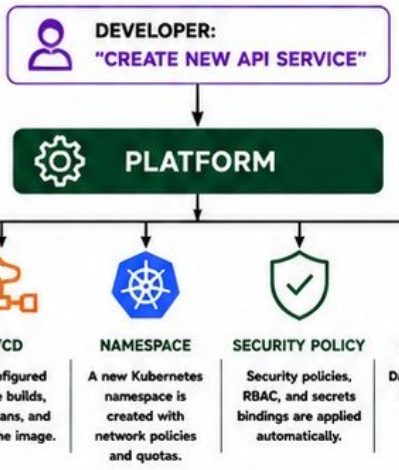
3 WHY NOT TICKETS FOR EVERYTHING?

- Tickets create bottlenecks and slow teams down.
- Manual work leads to inconsistencies.
- Developers lose context switching between building and waiting.
- Self-service enables speed, autonomy, and productivity.

4 WHAT THE PLATFORM PROVIDES

	**STANDARD APPLICATION TEMPLATES** Pre-built, opinionated foundations for common application types.
	**ENVIRONMENT PROVISIONING** Create dev, test, staging, or production environments on demand.
	**DATABASE REQUESTS** Request and provision databases with approved patterns and configurations.
	**KUBERNETES NAMESPACE CREATION** Create isolated namespaces with required policies and quotas.
	**CI/CD TEMPLATES** Reusable pipelines for build, test, scan, and deploy.
	**GUARDRAILS VS UNRESTRICTED ACCESS** Guardrails enforce security, compliance, and best practices while allowing developer freedom.

5 EXAMPLE: CREATE A NEW API SERVICE



RESULT: YOUR API SERVICE IS LIVE, SECURE, AND OBSERVABLE—WITHOUT A TICKET!

6 SELF-SERVICE PRINCIPLES

- Automate everything repeatable.
- Provide safe defaults.
- Make the easy path the right path.
- Give freedom within guardrails.
- Provide visibility and auditability.
- Enable teams, don't block them.
- Continuously improve the platform.

JUNIOR ENGINEER TIP

A great platform makes the right way easy and the wrong way hard. Self-service with guardrails is the key to scale.

DEVELOPER PORTALS AND BACKSTAGE

Developer portals are the front door to your Internal Developer Platform. They help developers discover, use, and operate everything they need.

1 WHAT IS A DEVELOPER PORTAL?

A developer portal is a centralized interface where developers can discover, access, and use platform services, tools, and documentation without needing to ask the platform team.

2 BACKSTAGE INTRODUCTION

Backstage

Backstage is an open platform from Spotify that helps you build a modern developer portal quickly.
It provides a framework, plugins, and tools to create a portal that fits your organization.

3 KEY FEATURES OF A DEVELOPER PORTAL

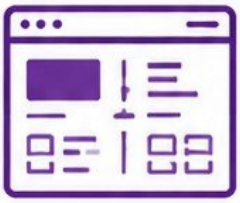
SOFTWARE CATALOG	A centralized list of all services, APIs, libraries, and components.
SERVICE OWNERSHIP	Clearly shows who owns a service and how to contact them.
TEMPLATES	Standardized templates to create new services, repositories, or environments.
DOCUMENTATION	Central place for guides, runbooks, policies, and how-to docs.
APIS	Self-service APIs to automate platform and infrastructure tasks.
PLUGINS	Extend functionality with plugins for CI/CD, monitoring, chatops, and more.

4 HOW A DEVELOPER USES THE PORTAL

5 DEVELOPER PORTAL vs INTERNAL DEVELOPER PLATFORM

DEVELOPER PORTAL		INTERNAL DEVELOPER PLATFORM (IDP)
The user interface for developers.		The combination of people, processes, tools, and platforms that deliver capabilities.
What developers see and interact with.		What makes everything work behind the scenes.
Built with tools like Backstage.		Includes CI/CD, Kubernetes, APIs, workflows, policies, databases, and more.
Focuses on experience and usability.		Focuses on automation, self-service, reliability, and governance.
A window.		The entire building.

6 WHY A PORTAL IS THE INTERFACE, NOT NECESSARILY THE ENTIRE PLATFORM



- The portal is what developers interact with.
- The platform includes many systems and services behind the scenes.
- A great portal can exist on top of a complex platform.
- The platform provides capabilities.
- The portal makes those capabilities easy to discover and use.

7 WHY DEVELOPER PORTALS MATTER

JUNIOR ENGINEER TIP

A powerful portal with good documentation, templates, and clear ownership makes developers love the platform and use it daily.

SECRETS, IDENTITY, AND PLATFORM SECURITY

Platform security protects your systems, data, and users while enabling developers to move fast and ship safely.

1 AUTHENTICATION vs AUTHORIZATION

- **AUTHENTICATION:** Proving who you are.
- **AUTHORIZATION:** Determining what you are allowed to do.

2 IAM (IDENTITY AND ACCESS MANAGEMENT)

- Manage users, groups, roles, and permissions.
- Control access to cloud resources and platform services.

3 RBAC (ROLE-BASED ACCESS CONTROL)

- Assign roles to users or service accounts.
- Grant only the permissions needed to do the job.

4 SERVICE ACCOUNTS

- Used by applications and workloads to access APIs.
- Avoid using human accounts for automation.

5 LEAST PRIVILEGE

- Give minimal permissions required.
- Reduce the blast radius if something goes wrong.

6 SECRETS

- Credentials, tokens, passwords, API keys, certificates, etc.
- Must be protected at all times.

7 WHY SECRETS SHOULD NOT BE HARDCODED

- Hardcoded secrets can be exposed in code.
- Hard to rotate and manage.
- Increases the risk of data breaches.
- Violates security and compliance rules.

8 SECRET MANAGERS

- Store, access, rotate, and audit secrets securely.
- Examples: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager.

9 WORKLOAD IDENTITY

- Workloads authenticate to cloud services without long-lived credentials.
- Examples: AWS IRSA, Azure Workload Identity, GCP Workload Identity.

10 POLICY ENFORCEMENT

- Enforce rules for security, compliance, and operations.
- Examples: OPA / Gatekeeper, Kyverno, Cloud Policies.

11 SECURITY GUARDRAILS

- Guardrails set safe boundaries for developers.
- Prevent risky actions while allowing flexibility within approved paths.

KEY SECURITY PILLARS

IDENTIFY WHO YOU ARE | CONTROL ACCESS | PROTECT SECRETS | ENFORCE POLICIES | MONITOR AND AUDIT | IMPROVE CONTINUOUSLY

SECURITY REMINDER

DEVELOPER CONVENIENCE + SECURITY GUARDRAILS = GOOD PLATFORM DESIGN

Balance speed and security to build a trusted and productive platform.

JUNIOR ENGINEER TIP

Security is NOT about slowing down developers. It is about enabling them to build and ship with confidence.

OBSERVABILITY AND PLATFORM RELIABILITY

Observability is how we understand what is happening inside our systems. It turns unknowns into answers and helps us build reliable platforms.

1 THE FOUR PILLARS OF OBSERVABILITY

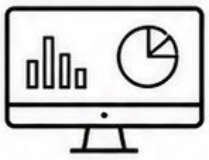
METRICS
- Numeric data over time.
- Shows WHAT is happening.
- Examples: CPU usage, request rate, error rate.

LOGS
- Timestamped records.
- Shows WHAT happened.
- Examples: application logs, access logs, system logs.

TRACES
- Request journey across services.
- Shows WHY something is slow or failing.
- Examples: latency, span details.

ALERTS
- Notifies when something is wrong.
- Based on metrics, logs, or traces.
- Helps teams respond fast.

2 DASHBOARDS



- Visualize your metrics, logs, and traces.
- Give instant insight into system health.
- Help teams monitor, analyze, and decide.

3 KEY OBSERVABILITY TOOLS

PROMETHEUS
Metrics collection and time-series database.

GRAFANA
Visualization and dashboarding platform.

LOKI
Log aggregation designed to work with Prometheus.

OPENTELEMETRY
Collects metrics, logs, and traces in a standard way.

ELASTICSEARCH
Centralized logging and search platform.

4 CENTRALIZED LOGGING

- Collect logs from all services and infrastructure.
- One place to search, analyze, and troubleshoot.
- Improves visibility and auditability.

5 APPLICATION HEALTH

- Health checks (liveness, readiness) ensure applications are running well.
- Monitor response time, error rate, and user impact.
- Understand how your application behaves from the outside and inside.

6 PLATFORM HEALTH

- Monitor Kubernetes, nodes, network, storage, and services.
- Ensure the platform itself is stable and reliable.
- A healthy platform means healthy applications.

7 WHY IT MATTERS

A platform must help developers answer the most important question:

"Why is my service failing?"

Observability provides the data. Understanding provides the answers.

Answers lead to faster fixes and better reliability.

8 OBSERVABILITY FLOW

APPLICATIONS
Generate telemetry (metrics, logs, traces)

→

COLLECTORS / AGENTS
Collect and forward telemetry

→

OBSERVABILITY BACKEND
Store, index, and process data

→

DASHBOARDS & ALERTS
Visualize, alert, and notify teams

→

ENGINEERS
Investigate issues and improve systems

9 BEST PRACTICES

- Instrument your applications.
- Monitor RED metrics (Rate, Errors, Duration).
- Use meaningful logs.
- Set smart alerts (not too noisy).
- Correlate metrics, logs, and traces.
- Create useful dashboards.
- Continuously review and improve.

JUNIOR ENGINEER TIP

You can't fix what you can't see. Observability is your superpower as a platform engineer.

PLATFORM TROUBLESHOOTING

A systematic approach helps you find the real problem faster and prevent it from happening again.

1 TROUBLESHOOTING FLOW

1. **SYMPTOM**
   What is the user or system experiencing?
2. **APPLICATION**
   Is the application behaving correctly?
3. **POD**
   Are the pods running as expected?
4. **KUBERNETES**
   Is the cluster and resources healthy?
5. **NETWORK**
   Is connectivity and DNS working?
6. **INFRASTRUCTURE**
   Are the nodes, storage, and load balancers OK?
7. **CLOUD SERVICE**
   Is the underlying cloud service available?

2 COMMON SCENARIOS

- **DEPLOYMENT FAILED**
  The deployment does not complete successfully.
- **POD IS CRASHLOOPBACKOFF**
  The container starts but keeps crashing.
- **IMAGE CANNOT BE PULLED**
  The image is missing or registry access fails.
- **DNS FAILS**
  DNS name does not resolve or is incorrect.
- **SERVICE CANNOT BE REACHED**
  The service is unreachable from inside or outside.
- **CI PIPELINE FAILS**
  Build, test, or deploy stage fails.
- **TERRAFORM FAILS**
  Infrastructure provisioning or update fails.
- **APPLICATION HAS NO SECRET**
  Required secrets are missing or incorrect.
- **DEVELOPER LACKS PERMISSION**
  Access is denied to resources or actions.

3 CORE PRINCIPLE: THE TROUBLESHOOTING METHOD

OBSERVE → **GATHER EVIDENCE** → **HYPOTHESIZE** → **TEST** → **FIX** → **VERIFY**

- **OBSERVE**: Look at the symptom and understand the impact.
- **GATHER EVIDENCE**: Collect logs, events, metrics, and any related information.
- **HYPOTHESIZE**: What could be causing this problem?
- **TEST**: Validate your theory with commands or changes.
- **FIX**: Apply the correct solution to resolve the root cause.
- **VERIFY**: Confirm the issue is resolved and monitor to prevent recurrence.

4 USEFUL TOOLS & COMMANDS

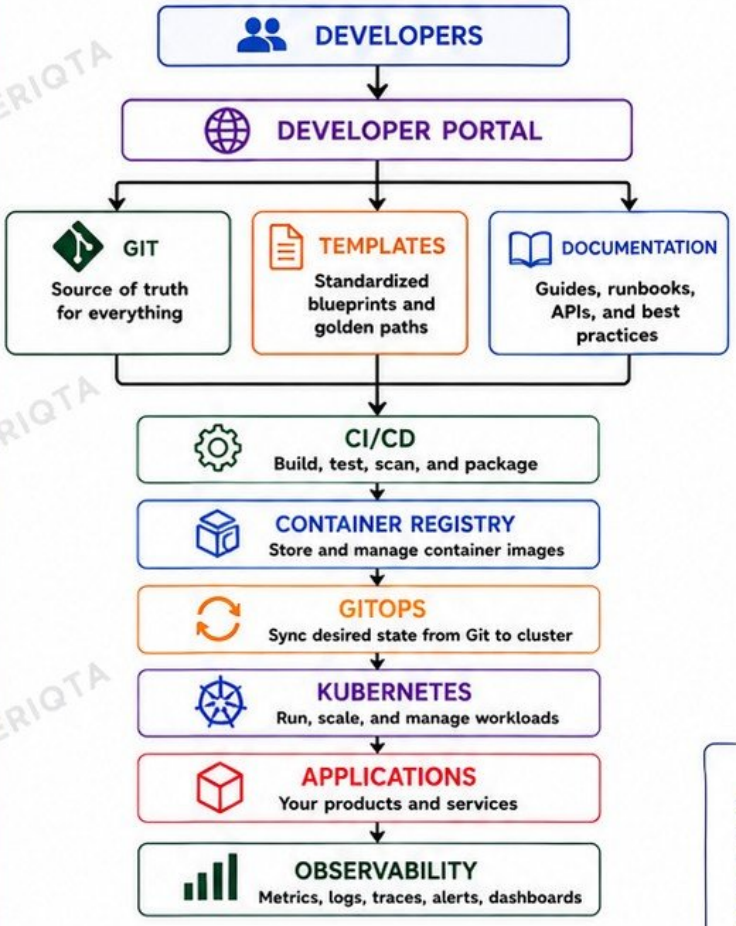
kubectl get all	kubectl logs	kubectl describe	kubectl exec	curl / ping / nslookup	cloud cli
Check resources	View logs	Detailed events	Debug inside pod	Network tests	Check cloud services

5 JUNIOR ENGINEER TIP

Don't guess. Follow the flow.
Go layer by layer.
The problem is usually closer to the top.
Evidence beats assumptions.

DESIGNING YOUR FIRST DEVELOPER PLATFORM

A simple, real-world architecture for an Internal Developer Platform (IDP) that helps teams build, deploy, and operate with speed and confidence.



SUPPORTING PLATFORM

TERRAFORM
Infrastructure as Code.
Provision and manage cloud infrastructure consistently.

IAM (IDENTITY & ACCESS MANAGEMENT)
Manage users, roles, and permissions.
Control access to cloud and platform resources.

SECRETS
Store and manage credentials, API keys, certificates, and sensitive information securely.

POLICIES & GUARDRAILS
Enforce standards, security, compliance, and cost controls across the platform.

WHY THIS ARCHITECTURE WORKS

- Developers get a self-service experience.
- Standardization reduces snowflakes.
- GitOps ensures consistency and reliability.
- Observability provides visibility and fast troubleshooting.
- Security and guardrails are built in by design.
- The platform scales with your organization.

WHO OWNS WHAT?

PLATFORM TEAM OWNS

- Developer portal and self-service experience
- Templates, golden paths, and documentation
- CI/CD templates and shared pipelines
- GitOps systems and cluster operations
- Platform tools, integrations, and plugins
- Infrastructure (via Terraform)
- Security, policies, and guardrails
- Observability platforms and alerts
- Secrets and access management
- Enablement, training, and developer support

COLLABORATION DELIVERS VALUE

APPLICATION TEAMS OWN

- Application source code and repositories
- Service configuration and environments
- Business logic and application dependencies
- Service-level objectives (SLOs) for their services
- Data they create and manage
- Application-specific monitoring and alerts
- Cost optimization within their services
- Incident response for their applications
- Continuous improvement of their services

JUNIOR ENGINEER TIP: A platform is not just infrastructure. It is the combination of people, processes, tools, and automation that makes developers successful.

YOUR GOAL: Make the right path the easy path.

EMPOWER DEVELOPERS. ELEVATE EVERYONE.

FINAL BEGINNER PLATFORM ENGINEERING PROJECT

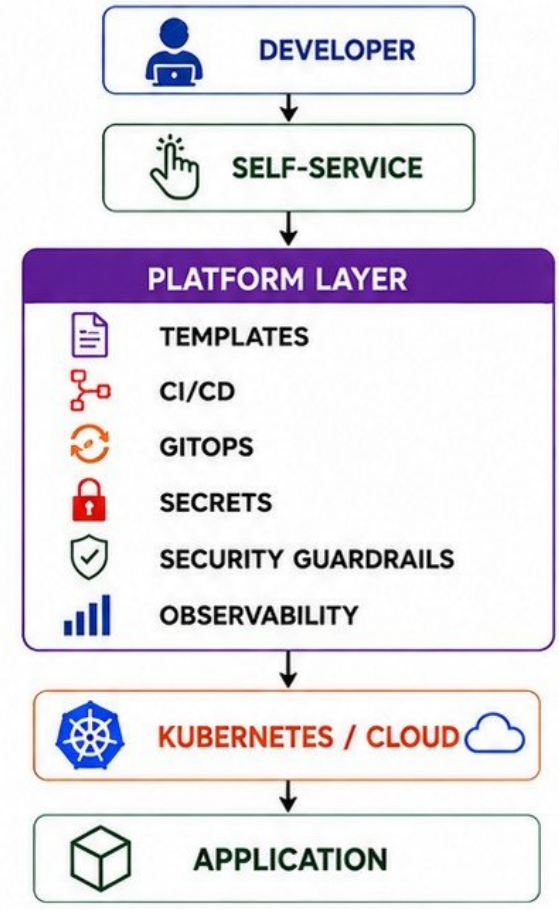
BUILD YOUR FIRST MINI INTERNAL DEVELOPER PLATFORM

In this final project, you will bring everything together to build, deploy, and operate a mini Internal Developer Platform.

1 PROJECT REQUIREMENTS

1. Create an application repository
2. Containerize a simple application
3. Store it in Git
4. Create Terraform infrastructure
5. Provision a Kubernetes environment
6. Create reusable Kubernetes or Helm configuration
7. Build a CI pipeline
8. Push the image to a registry
9. Deploy through GitOps
10. Configure application secrets safely
11. Add health checks
12. Add resource requests and limits
13. Add basic monitoring
14. Create a simple service template
15. Deploy a second application using the same platform path
16. Introduce a deployment failure
17. Troubleshoot it
18. Recover the service
19. Document the golden path

2 FINAL MENTAL MODEL



3 WHAT YOU WILL LEARN

- End-to-end platform thinking
- Automating infrastructure and delivery
- GitOps and declarative operations
- Secure secret management
- Self-service and developer experience
- Observability and reliability
- Troubleshooting and recovery
- Documentation and golden path creation

4 SUCCESS CRITERIA

- Two applications deployed successfully
- Secure secrets management
- Pipeline automation working
- GitOps deployments in place
- Monitoring and health checks enabled
- Failure introduced and resolved
- Golden path documented

5 JUNIOR ENGINEER TIP

Build small.
Automate everything.
Document clearly.
Fix problems.
Then you are thinking like a Platform Engineer.

YOU JUST BUILT YOUR FIRST MINI INTERNAL DEVELOPER PLATFORM.

KEEP ITERATING. KEEP IMPROVING. KEEP BUILDING!